Animal Management — CRUD Lookup Table

Branch: `main`
Commit: `8c351f0077d49d4523567ec7867c6953bba5bdfb`

1. Requirement-to-CRUD Lookup

Requirement	Create	Read	Update	Delete
Register animal	✓			
Display all animals		✓		
View animal details		✓		
Edit animal information			✓	
Delete animal				✓
Change adoption availability			✓	

2. Implementation Mapping

User operation in Animal screen	Controller action	Model	Repository call	CRUD
Register animal	`AnimalController.addAnimal()`	`Animal`	`AnimalRepository.save(animal)`	C
Display all animals	`AnimalController.refreshTable()`	`Animal`	`AnimalRepository.findAll()`	R
View animal details	`AnimalController.loadSelectedAnimal()`	`Animal`	`AnimalRepository.findById(id)`	R
Edit animal information	`AnimalController.updateAnimal()`	`Animal`	`AnimalRepository.save(updatedAnimal)`	U
Delete animal	`AnimalController.deleteAnimal()`	`Animal`	`AnimalRepository.delete(id)`	D
Change adoption availability	`AnimalController.updateAnimal()`	`Animal`	`AnimalRepository.save(updatedAnimal)`	U

3. Components

- **`AnimalController`** — handles user actions from `AnimalView` and calls the repository.
- **`AnimalRepository`** — extends `LookupTable<Animal>`; backed by the in-memory `DataStore`.
- **`Animal`** — model with `id`, `name`, `species`, `ageYears`, `ageMonths`, `healthStatus`, `availableForAdoption`, `description`.

4. Notes

- Species and health status are selected from combo boxes, but the model stores them as strings.
- Change adoption availability is performed through the edit flow (`updateAnimal()`); it is not a separate controller method.
- `LookupTable<T>` is a generic repository implementation, not a dedicated editable lookup-data screen.
- Data is held in memory and is lost when the application closes.